

BOOK 2 — Atlas Architecture & System Specifications

Document Type: Software Architecture & System Specification (SASS)

Version: 0.1

Status: Engineering Draft

Classification: Internal Engineering Documentation

Depends On: Book 1 – Atlas Foundation

CHAPTER 1

Software Requirements Specification (SRS)

1.1 Document Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements of the Atlas platform.

Unlike Book 1, which establishes the vision, philosophy, and product definition of Atlas, this document specifies the engineering requirements that guide implementation.

The purpose of this SRS is to ensure that every subsystem of Atlas is developed according to a consistent, traceable, and verifiable specification.

This document serves as the primary reference for software engineers, architects, testers, and future contributors responsible for implementing Atlas.

The requirements documented herein are technology-independent wherever possible. Architectural implementation details are described in later chapters of this book.

1.2 Scope

Atlas is an AI Evaluation Operating System that provides standardized infrastructure for creating, executing, evaluating, and reporting AI benchmarks.

The scope of Version 1 includes:

- Benchmark definition and management
- Benchmark versioning
- Execution through modular execution adapters
- Evaluation using configurable evaluation strategies
- Reporting and analytics

- Dashboard for user interaction
- Authentication and project management

Capabilities explicitly excluded from Version 1 include marketplace functionality, benchmark monetization, enterprise billing, adaptive benchmark mutation, and automated benchmark generation.

1.3 Intended Audience

This document is intended for:

- Software Engineers
- System Architects
- Backend Developers
- Frontend Developers
- Machine Learning Engineers
- Researchers
- QA Engineers
- Technical Writers
- Future Contributors

Each audience should use this document as the authoritative source for implementation requirements.

1.4 Document Conventions

To maintain consistency across the Atlas Engineering Knowledge Base, all requirements shall use standardized identifiers.

- **Functional Requirements:** FR-XXX-001 (Example: FR-BE-001 Create Benchmark)
- **Non-Functional Requirements:** NFR-XXX-001 (Example: NFR-SEC-001 Encrypted Credential Storage)
- **Architecture Decision Records:** ADR-001 (Atlas is Infrastructure)
- **Research Tasks:** RS-001 (Benchmark Evolution Research)
- **Future Features:** FF-001 (Marketplace)

1.5 Definitions

Term	Definition
Atlas Run	One complete execution of a benchmark against a model
Atlas Benchmark	A versioned benchmark artifact
Atlas Report	A structured evaluation report

Execution Adapter	Component responsible for communicating with AI systems
Evaluation Strategy	Algorithm used to score outputs
Benchmark Version	Immutable revision of a benchmark
Capability Profile	Multi-dimensional representation of evaluation results

1.6 References

This document references:

- Book 1 — Atlas Foundation
- IEEE 29148 — Software Requirements Specification
- Architecture Decision Records
- Atlas Engineering Standards
- Atlas Research Register

1.7 Overview

The remainder of this book defines:

- Complete system requirements
- Architecture
- APIs
- Database
- Security
- Infrastructure
- Engineering standards
- Build order

Together these chapters form the implementation blueprint for Atlas.

CHAPTER 2

Atlas System Decomposition

Chapter Status

Property	Value
Status	Approved

Engineering Stage	System Design
Priority	Critical
Owner	Architecture Team
Related ADR	ADR-001, ADR-003

2.1 Purpose

Before software can be implemented, the engineering team must identify every subsystem that composes the Atlas platform.

Rather than viewing Atlas as a single application, this document treats Atlas as a collection of independent but cooperating systems.

Each subsystem owns a clearly defined responsibility and communicates with other subsystems through documented interfaces.

This decomposition minimizes coupling, improves maintainability, and allows individual components to evolve independently.

2.2 Atlas as a System of Systems

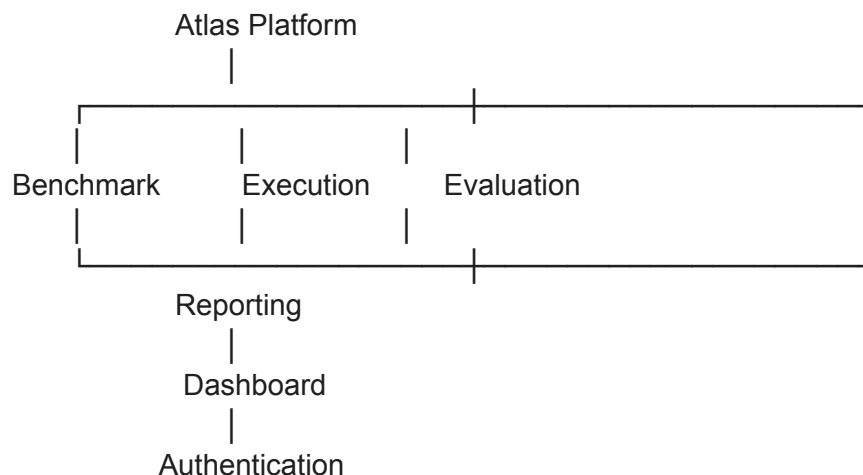
Atlas is not a monolithic application.

Instead, Atlas is a System of Systems (SoS).

Each subsystem performs one primary responsibility.

Together these systems implement the complete AI evaluation lifecycle.

Conceptually:



2.3 Core Subsystems

Atlas Version 1 consists of the following major engineering subsystems.

ID	Subsystem	Priority
SYS-001	Benchmark Engine	Critical
SYS-002	Execution Engine	Critical
SYS-003	Evaluation Engine	Critical
SYS-004	Reporting Engine	Critical
SYS-005	Dashboard	High
SYS-006	Authentication Service	High
SYS-007	Project Management	Medium
SYS-008	Dataset Registry	Medium
SYS-009	Audit & Logging	High
SYS-010	Configuration Service	High

These subsystems collectively implement Atlas Version 1.

2.4 Benchmark Engine

Purpose

Own the lifecycle of benchmarks.

Responsibilities

- Create benchmarks.
- Validate benchmark definitions.
- Store benchmark metadata.
- Manage benchmark versions.
- Publish benchmarks.

- Archive deprecated benchmarks.

Inputs

- Benchmark definitions
- Metadata
- Test cases

Outputs

- Versioned benchmark artifacts

Dependencies

- Database
- Authentication
- Dataset Registry

2.5 Execution Engine

Purpose

Execute benchmarks against AI systems.

Responsibilities

- Receive execution requests.
- Select execution adapter.
- Schedule execution.
- Retry failures.
- Collect outputs.
- Store execution logs.

Inputs

- Benchmark
- Model
- Configuration

Outputs

- Raw execution results

Dependencies

- Benchmark Engine
- Adapter Framework
- Queue Service

2.6 Evaluation Engine

Purpose

Evaluate execution results.

Responsibilities

- Hidden-test grading.
- Metric calculation.
- Score generation.
- Capability analysis.
- Evaluation metadata.

Inputs

- Model output
- Benchmark
- Ground truth

Outputs

- Evaluation Report

Dependencies

- Execution Engine

2.7 Reporting Engine

Purpose

Convert evaluation results into structured reports.

Responsibilities

- Generate reports.
- Compare historical runs.
- Produce capability profiles.
- Export reports.

Outputs

- JSON
- PDF
- Dashboard summaries

2.8 Dashboard

Purpose

Provide user interaction.

Capabilities

- Browse benchmarks
- Create projects

- Start evaluations
- Compare models
- View reports

The Dashboard never performs evaluation itself.

It communicates exclusively through backend APIs.

2.9 Authentication Service

Responsibilities

- Login
- Registration
- Sessions
- Roles
- Permissions
- API Tokens

Future versions may support enterprise SSO providers.

2.10 Dataset Registry

Purpose

Manage every dataset known to Atlas.

Responsibilities

- Dataset metadata
- Licensing
- Version history
- Provenance
- Validation

The Dataset Registry stores metadata only.

Actual datasets may reside in external storage.

2.11 Audit Service

Every important action performed within Atlas should be recorded.

Examples include:

- Benchmark creation
- Benchmark updates
- Evaluation execution
- User authentication
- Report generation

Audit logs support reproducibility and debugging.

2.12 Configuration Service

Purpose

Centralize platform configuration.

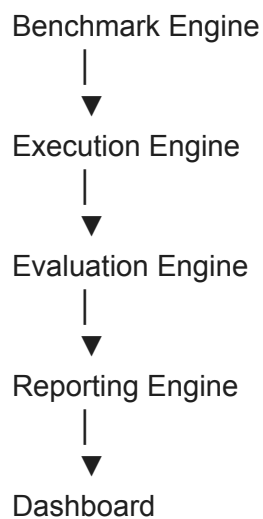
Stores

- Environment settings
- Feature flags
- Execution defaults
- Adapter configuration
- Database settings

No subsystem should contain hard-coded configuration.

2.13 System Relationships

The interaction between major subsystems follows a defined workflow.



Authentication, Configuration, and Audit services support every stage of this workflow.

2.14 Engineering Principles

Every subsystem should satisfy the following principles:

- Single Responsibility
- Loose Coupling
- High Cohesion
- Interface Stability
- Independent Testing

- Version Control
- Horizontal Scalability (Future)

2.15 Builder Notes

Property	Value
Difficulty	High
Prototype Required	Yes
Estimated Duration	3 Weeks
Owner	Backend Team
Research Required	Low
ML Required	No

Engineering Notes

Confirmed Decisions

- Atlas is composed of independent engineering subsystems.
- Each subsystem owns one primary responsibility.
- Communication occurs through documented interfaces.
- The Dashboard is a presentation layer, not a business logic layer.

Open Questions

- Should the Configuration Service support remote configuration?
- Is the Dataset Registry a standalone microservice in future versions?
- Should Report Generation become asynchronous?

CHAPTER 3

Functional Requirements Specification

Document Status

Property	Value
Status	Approved
Engineering Phase	Requirements Engineering

Priority	Critical
Owner	Atlas Engineering Team
Classification	Functional Specification
Related Books	Book 1, Book 3, Book 4

3.1 Purpose

The Functional Requirements Specification defines the observable behavior of every Atlas subsystem.

Each requirement describes what the system must do, independent of how it will eventually be implemented.

These requirements become the contractual agreement between product design, software architecture, implementation, and testing.

Every implemented feature within Atlas must trace back to one or more functional requirements documented in this chapter.

3.2 Requirement Classification

Atlas organizes functional requirements into subsystem groups.

Prefix	Subsystem
FR-BE	Benchmark Engine
FR-EX	Execution Engine
FR-EV	Evaluation Engine
FR-RP	Reporting Engine
FR-DB	Dashboard
FR-AU	Authentication
FR-DS	Dataset Registry
FR-CFG	Configuration

FR-AD	Adapter Framework
-------	-------------------

Each requirement is uniquely identified.

Example: FR-BE-001

3.3 Benchmark Engine Requirements

FR-BE-001

Title: Create Benchmark

Description: The system shall allow authorized users to create a benchmark using the Atlas Benchmark Schema.

Priority: Critical

Inputs:

- Benchmark name
- Description
- Category
- Metadata
- Test cases
- Evaluation strategy

Outputs:

- Benchmark ID
- Version 1.0
- Creation timestamp

Acceptance Criteria:

- Benchmark validates successfully.
- Benchmark stored.
- Version generated.
- Audit log created.

FR-BE-002 Update Benchmark

The system shall allow benchmark authors to create new benchmark revisions while preserving historical versions.

Historical versions must remain immutable.

FR-BE-003 Publish Benchmark

Only validated benchmarks may be published.

Publishing creates a permanent benchmark version.

FR-BE-004 Archive Benchmark

Benchmarks may be archived without deletion.

Archived benchmarks remain reproducible.

FR-BE-005 Benchmark Search

Users shall search benchmarks by:

- Name
- Category
- Tags
- Version
- Author
- Difficulty

FR-BE-006 Benchmark Metadata

Every benchmark shall contain:

- Name
- Description
- Version
- Category
- License
- Creation date
- Last update
- Author
- Status

3.4 Execution Engine Requirements

FR-EX-001 Execute Benchmark

The system shall execute a benchmark against one or more configured models.

FR-EX-002 Execution Queue

Evaluation jobs shall be queued asynchronously.

FR-EX-003 Retry Failed Execution

Temporary execution failures shall automatically retry according to configurable retry policies.

FR-EX-004 Execution Logs

Every execution shall generate structured logs.

FR-EX-005 Execution Metadata

Each execution shall record:

- Model
- Adapter
- Timestamp
- Runtime
- Configuration
- Resource usage

3.5 Evaluation Engine Requirements

FR-EV-001 Hidden Test Evaluation

Support hidden test grading.

FR-EV-002 Metric Calculation

Automatically calculate evaluation metrics.

FR-EV-003 Capability Profile

Generate multidimensional capability profiles.

FR-EV-004 Store Evaluation Results

Evaluation results shall be permanently stored.

FR-EV-005 Evaluation Determinism

Repeated evaluations using identical inputs should produce equivalent results wherever technically possible.

3.6 Reporting Requirements

FR-RP-001 Generate Report

Produce evaluation reports after successful completion.

FR-RP-002 Export Formats

Support:

- JSON
- PDF
- CSV

Future:

- HTML

FR-RP-003 Historical Comparison

Compare multiple benchmark runs.

FR-RP-004 Capability Visualization

Display multidimensional capability summaries.

3.7 Dashboard Requirements

FR-DB-001 Dashboard Home

Display recent evaluations.

FR-DB-002 Benchmark Browser

Browse available benchmarks.

FR-DB-003 Project Dashboard

Display project information.

FR-DB-004 Evaluation History

Show historical runs.

FR-DB-005 Search

Search across:

- Projects
- Benchmarks
- Reports
- Runs

3.8 Authentication Requirements

FR-AU-001 User Registration

FR-AU-002 Login

FR-AU-003 Logout

FR-AU-004 Role Management

Roles include:

- Admin
- Benchmark Author
- Researcher
- Developer
- Viewer

FR-AU-005 Session Management

Secure session handling.

3.9 Dataset Registry Requirements

FR-DS-001 Register Dataset

FR-DS-002 Dataset Metadata

FR-DS-003 Dataset Versioning

FR-DS-004 License Tracking

FR-DS-005 Dataset Validation

3.10 Configuration Requirements

FR-CFG-001 Feature Flags

FR-CFG-002 Execution Settings

FR-CFG-003 Adapter Configuration

FR-CFG-004 Environment Profiles

- Development
- Testing
- Production

3.11 Adapter Framework Requirements

FR-AD-001 Adapter Discovery

The platform shall detect registered execution adapters.

FR-AD-002 Adapter Selection

Execution engine selects adapters dynamically.

FR-AD-003 Adapter Isolation

Adapter failures shall not crash Atlas Core.

FR-AD-004 Adapter Standard Interface

Every adapter shall implement the Atlas Adapter Contract.

3.12 Requirement Priority Matrix

Priority	Meaning
----------	---------

Critical	Required for Version 1
High	MVP Expansion
Medium	Version 2
Low	Future Research

3.13 Requirement Traceability

Every requirement shall trace to:

- Product Goal
- Subsystem
- API
- Database
- Test Cases
- Documentation
- Future Code Module

Example:

FR-BE-001

↓

Book 1 Goal G-001

↓

Benchmark Engine

↓

POST /benchmarks

↓

Benchmark Table

↓

Integration Test

↓

benchmark_service.py

3.14 Builder Notes

- **Estimated Functional Requirements Version 1:** $\approx$ 180–220
- **Version 2:** $\approx$ 350+

Every implemented feature must reference at least one requirement ID.

Engineering Notes

Confirmed Decisions

- Requirements drive implementation.
- Every subsystem owns independent functional requirements.
- Functional requirements are technology-independent.
- Requirements remain version controlled.

Research Questions

- Which requirements belong in MVP?
- Which become enterprise-only?
- Which remain future research?

CHAPTER 4

Overall System Architecture

Document Status

Property	Value
Status	Approved
Engineering Phase	High-Level Architecture
Priority	Critical
Owner	System Architecture Team
Classification	Architecture Specification
Related Books	Book 1, Book 3, Book 4

4.1 Purpose

The purpose of this chapter is to define the high-level architecture of the Atlas platform.

Rather than viewing Atlas as a single application, the platform is designed as a collection of independent services that cooperate through well-defined interfaces.

This architecture emphasizes:

- Modularity
- Separation of concerns
- Extensibility
- Scalability
- Reproducibility

- Maintainability

Every future implementation of Atlas shall conform to this architectural model unless formally superseded through an Architecture Decision Record (ADR).

4.2 Architectural Philosophy

Atlas adopts a Modular Layered Architecture.

Each layer has one responsibility.

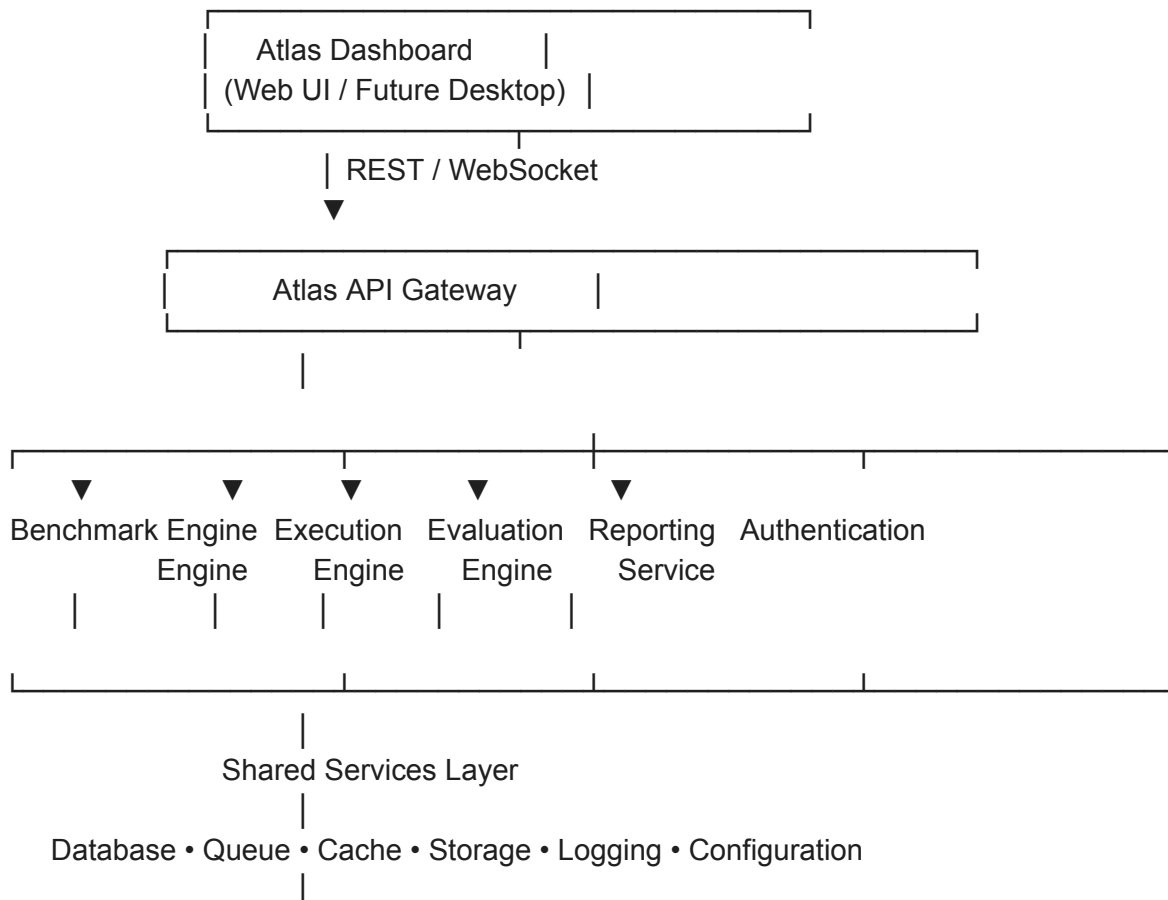
A layer may only communicate with adjacent layers through defined interfaces.

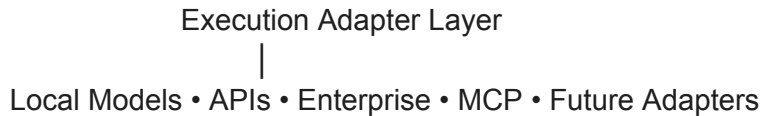
This minimizes coupling while allowing each subsystem to evolve independently.

The architecture follows four guiding principles:

1. **Single Responsibility** – Each subsystem owns one domain.
2. **Interface First** – Communication occurs through contracts, not direct implementation.
3. **Replaceability** – Any component can be replaced without redesigning the entire platform.
4. **Technology Independence** – Business logic is isolated from infrastructure.

4.3 High-Level Architecture





This architecture intentionally separates presentation, business logic, infrastructure, and execution backends.

4.4 Architectural Layers

Atlas is divided into seven architectural layers.

Layer 1 — Presentation Layer

Responsible for user interaction. Components include:

- Dashboard
- Authentication Pages
- Benchmark Browser
- Report Viewer
- Administration Panel

This layer contains no business logic. Its only responsibility is presenting information and collecting user input.

Layer 2 — API Layer

The API Layer acts as the gateway into Atlas. Responsibilities include:

- Request validation
- Authentication
- Authorization
- Rate limiting
- API versioning
- Request routing

All client communication passes through this layer. Future clients (CLI, desktop, mobile) will also communicate through the same API.

Layer 3 — Business Logic Layer

This is the core of Atlas. It contains:

- Benchmark Engine
- Execution Engine
- Evaluation Engine
- Reporting Engine

No UI code should exist here. No database-specific code should exist here. Business logic

remains independent of infrastructure.

Layer 4 — Infrastructure Services

Infrastructure services support every subsystem. Examples:

- Configuration Service
- Audit Service
- Logging
- Notification Service
- Background Workers
- Scheduler

These services are shared across the platform.

Layer 5 — Data Layer

Persistent storage. Components include:

- PostgreSQL
- Redis
- Object Storage
- Dataset Registry

This layer stores: Benchmarks, Reports, Executions, Users, Projects, Audit Logs.

Business logic must never depend directly on database implementation details.

Layer 6 — Adapter Layer

One of Atlas's defining architectural features. The Adapter Layer provides a standardized interface between Atlas and external AI systems. Supported adapters include:

- Local Model Adapter (Ollama)
- API Adapter (OpenAI-compatible endpoints)
- Enterprise Adapter
- MCP Adapter (future)
- Experimental Browser Adapter (research only)

Each adapter implements the same execution contract, ensuring that the core platform remains independent of model providers.

Layer 7 — External Systems

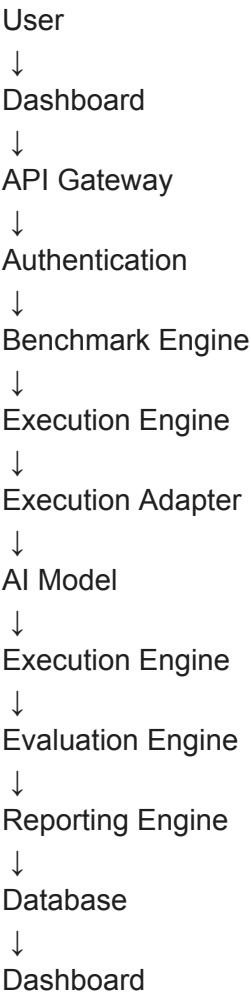
External systems are not controlled by Atlas but interact through adapters. Examples include:

- Local LLM runtimes
- Cloud inference APIs
- Enterprise AI deployments
- Future model providers

The architecture ensures these systems can change without affecting Atlas's internal logic.

4.5 Request Lifecycle

A typical evaluation request follows this sequence:



Each stage produces structured logs and metadata to maintain reproducibility.

4.6 Architectural Characteristics

The Atlas architecture is designed to satisfy the following quality attributes:

Attribute	Description
Modularity	Independent subsystems with defined responsibilities

Scalability	Support future horizontal scaling of services
Extensibility	New adapters and benchmark types can be added without redesign
Maintainability	Clear boundaries reduce engineering complexity
Reproducibility	Evaluation workflows are deterministic where possible
Observability	Logs, metrics, and traces are available across all services
Portability	Deployable on local machines, servers, or cloud environments

4.7 Service Communication

Version 1 of Atlas will use synchronous REST APIs for most interactions.

Background operations (benchmark execution, report generation) will use asynchronous workers connected through a task queue.

Future versions may introduce event-driven messaging for large-scale deployments.

4.8 Error Handling Strategy

Every subsystem shall implement a common error model.

Errors are classified as:

- Validation Errors
- Authentication Errors
- Authorization Errors
- Execution Errors
- Evaluation Errors
- Infrastructure Errors
- External Adapter Errors

All errors must include: Error Code, Human-readable Message, Timestamp, Correlation ID, Suggested Action (where applicable).

This enables consistent debugging and improves user experience.

4.9 Architectural Constraints

The architecture shall satisfy the following constraints:

- No subsystem may directly access another subsystem's database tables.
- All communication shall occur through documented interfaces.
- External AI providers shall only be accessed through adapters.
- Business logic shall remain independent of UI frameworks.
- Infrastructure components shall be replaceable with minimal impact.

4.10 Future Evolution

The architecture is designed to evolve without requiring major redesign. Potential future enhancements include:

- Distributed execution clusters
- Kubernetes orchestration
- Plugin marketplace
- Federated benchmark repositories
- Multi-region deployments
- AI-assisted benchmark authoring
- Real-time collaborative evaluation sessions

These capabilities build upon the existing layered architecture rather than replacing it.

Builder Notes

Property	Value
Difficulty	High
Prototype Required	Yes
Estimated Build Time	4 Weeks
Owner	Architecture & Backend Team
Dependencies	Chapter 2, Chapter 3
Research Required	Medium
ML Required	No

Engineering Notes

Confirmed Decisions

- Atlas uses a layered modular architecture.
- Execution adapters isolate external AI providers.

- Business logic is independent of presentation and infrastructure.
- Shared services provide cross-cutting capabilities.

Open Questions

- Should the API Gateway remain a single service or be split in future versions?
- When should asynchronous event messaging replace REST for internal communication?
- Which components should become independently deployable microservices?

CHAPTER 5

Core Services Architecture

Document Information

Property	Value
Status	Approved
Engineering Stage	Service Architecture
Priority	Critical
Owner	Backend Architecture Team
Depends On	Chapter 2, Chapter 3, Chapter 4
Related Books	Book 3, Book 4

5.1 Purpose

Atlas is composed of multiple independent services.

Each service owns a specific business capability and exposes that capability through documented interfaces.

The objective of this chapter is to define every core service, its responsibilities, its lifecycle, interfaces, ownership, dependencies, scalability strategy, and implementation boundaries.

This chapter intentionally avoids implementation code. Implementation is documented in Book 4.

5.2 Service-Oriented Architecture

Atlas follows a Modular Service-Oriented Architecture (SOA).

Each service owns one business domain. No service may directly manipulate another service's

internal state.

Communication occurs through:

- REST APIs
- Internal service contracts
- Event notifications (future)
- Shared domain models

Every service is independently testable.

5.3 Atlas Core Services

Version 1 consists of ten primary services.

ID	Service	Responsibility
SVC-001	Benchmark Service	Benchmark lifecycle
SVC-002	Execution Service	Model execution
SVC-003	Evaluation Service	Result evaluation
SVC-004	Reporting Service	Report generation
SVC-005	Authentication Service	Users & security
SVC-006	Project Service	Project management
SVC-007	Dataset Service	Dataset registry
SVC-008	Configuration Service	Platform configuration
SVC-009	Audit Service	Logging & traceability
SVC-010	Notification Service	Alerts & messaging

5.4 Benchmark Service

Purpose: The Benchmark Service owns the complete lifecycle of every benchmark within Atlas. It is the authoritative source for benchmark definitions.

Responsibilities:

- Create benchmark
- Update benchmark
- Validate schema

- Manage versions
- Archive benchmarks
- Publish benchmarks
- Manage metadata
- Assign categories
- Store benchmark assets

Inputs: Benchmark Definition, Test Cases, Metadata, Evaluation Strategy

Outputs: Benchmark Artifact, Benchmark Version, Validation Report

Internal Components:

Benchmark Controller → Validation Manager → Version Manager → Metadata Manager → Repository Layer

Database Ownership:

The Benchmark Service exclusively owns: benchmarks, benchmark_versions, benchmark_categories, benchmark_assets.

No other service writes directly to these tables.

5.5 Execution Service

Purpose: The Execution Service manages communication between Atlas and AI systems. It never evaluates results. It only executes requests.

Responsibilities:

- Queue executions
- Select adapters
- Execute models
- Retry failures
- Capture raw outputs
- Record execution metadata

Internal Components:

Execution Controller → Queue Manager → Adapter Manager → Execution Worker → Execution Repository

State Machine:

Created → Queued → Running → Completed → Evaluated

Possible failure states: Timeout, Adapter Failure, Network Failure, User Cancellation

5.6 Evaluation Service

Purpose: Transform raw execution outputs into structured evaluation results.

Responsibilities:

- Hidden test grading
- Score calculation
- Metric aggregation
- Capability analysis
- Consistency checking

Inputs: Raw Model Output, Benchmark, Ground Truth

Outputs: Evaluation Score, Capability Profile, Evaluation Metadata

Future Expansion: Future versions may support LLM Judge, Human Judge, Hybrid Judge, Confidence Estimation. These are explicitly outside Version 1.

5.7 Reporting Service

Purpose: Produce reproducible reports from evaluation data.

Responsibilities: PDF generation, JSON export, CSV export, Historical comparison, Capability summaries, Leaderboards.

Internal Pipeline:

Evaluation Results → Report Builder → Template Engine → Formatter → Exporter

5.8 Authentication Service

Responsibilities: Registration, Login, Logout, Password Reset, API Tokens, Role Management, Sessions.

Roles: Administrator → Researcher → Developer → Benchmark Author → Viewer. (Future: Enterprise Roles).

5.9 Project Service

Projects organize evaluations. Every project contains Models, Benchmarks, Reports, Executions, Members. Projects become the primary collaboration unit.

5.10 Dataset Service

Purpose: Manage dataset metadata (not storage).

Responsibilities: Dataset registration, Versioning, Provenance, Licensing, Validation. (Future: Dataset marketplace).

5.11 Configuration Service

Central configuration. Stores Environment, Feature Flags, Adapter Config, Limits, Defaults. Every service reads configuration here. No hardcoded configuration.

5.12 Audit Service

Atlas is built around reproducibility. Every important action must be auditable (Login, Benchmark creation, Evaluation, Report generation, Configuration changes).

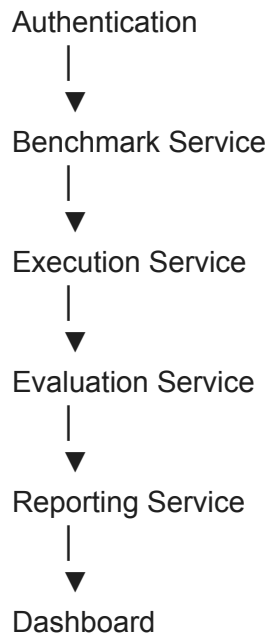
Every audit event contains Timestamp, User, Resource, Action, Previous State, New State.

5.13 Notification Service

Version 1: Simple notifications (Evaluation completed, Benchmark published, Execution failed).

Future: Email, Slack, Teams, Discord, Webhooks.

5.14 Service Dependencies



Supporting Services: Configuration, Dataset, Audit, Notifications.

5.15 Service Contracts

Every service shall expose: REST API, OpenAPI Specification, Version Number, Health Endpoint, Metrics Endpoint.

Every service shall return: Standard Errors, Correlation IDs, Structured Responses.

5.16 Service Quality Attributes

Attribute	Requirement
Availability	High
Scalability	Medium (V1)
Maintainability	Critical
Testability	Critical
Replaceability	Critical
Security	Critical
Observability	High

5.17 Failure Strategy

Every service must degrade gracefully. If one service fails, Atlas should isolate the failure, preserve audit logs, notify users, retry where appropriate, and avoid cascading failures.

5.18 Builder Notes

Property	Value
Build Order	After Database
Difficulty	High
Owner	Backend Team
Estimated Duration	5 Weeks
Prototype Required	Yes
ML Required	No

Engineering Notes

Confirmed Decisions:

- Atlas uses service-oriented architecture.
- Every service owns its own business domain.
- Services communicate through documented interfaces.

- Services are independently testable.
- Database ownership follows bounded contexts.

Open Questions:

- Should Notification Service become asynchronous in V2?
- Should Project Service evolve into a collaboration platform?
- Which services should eventually become standalone microservices?

CHAPTER 6

Execution Adapter Architecture

Document Information

Property	Value
Status	Approved
Engineering Phase	Architecture
Priority	Critical
Owner	Platform Architecture Team
Depends On	Chapter 4, Chapter 5
Related Books	Book 3, Book 4

6.1 Purpose

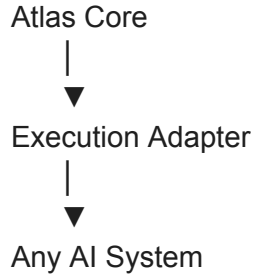
The Execution Adapter Layer is the abstraction layer that separates Atlas Core from external Artificial Intelligence systems. Rather than embedding provider-specific logic inside the Benchmark or Execution Engines, Atlas communicates exclusively through standardized adapters.

This architectural decision ensures that the core evaluation platform remains independent of AI providers, APIs, local inference engines, enterprise deployments, and future execution technologies. As AI ecosystems evolve, only adapters require modification while Atlas Core remains unchanged.

6.2 Why an Adapter Architecture?

Without adapters, every subsystem would need to understand the unique behavior of each provider. This creates tight coupling and duplicated logic. Instead, Atlas introduces a unified

execution contract:



Atlas Core never knows which provider executes the request.

6.3 Architectural Principles

Every adapter must satisfy the following principles:

- **Provider Independence:** Atlas Core must never contain provider-specific logic.
- **Interface Consistency:** Every adapter exposes the same public methods regardless of provider.
- **Replaceability:** Removing or adding an adapter must not require modifications to Atlas Core.
- **Fault Isolation:** Failure inside one adapter shall not affect the execution of other adapters.
- **Extensibility:** Future adapters should be added through registration rather than modification.

6.4 Adapter Types

Version 1 officially supports three execution methods.

Adapter	Status	Purpose
Local Adapter	MVP	Ollama / vLLM
API Adapter	MVP	OpenAI-compatible APIs
Enterprise Adapter	Future	Internal enterprise endpoints

Research adapters: MCP Adapter, Browser Adapter, Batch Execution Adapter, Distributed Execution Adapter.

6.5 Standard Adapter Interface

Every adapter must implement the Atlas Adapter Contract.


```
class ExecutionAdapter:
    def connect()
    def validate()
    def execute()
    def cancel()
    def health_check()
    def disconnect()
```

This interface remains stable even as implementations evolve.

6.6 Adapter Lifecycle

Initialize → Validate Configuration → Connect → Execute Prompt → Receive Output → Return Standard Response → Disconnect.

Errors at any stage must be propagated using Atlas Error Codes.

6.7 Adapter Registration

Atlas discovers adapters through a registry. Every adapter receives: Adapter ID, Version, Supported Models, Capabilities, Health Status.

6.8 Adapter Metadata

Each registered adapter exposes metadata. Example:

Property	Example
Adapter ID	local-ollama
Version	1.0
Provider	Ollama
Execution Mode	Local
Streaming	Supported
Max Context	Configurable
Status	Healthy

6.9 Execution Flow

User → Benchmark → Execution Engine → Adapter Registry → Selected Adapter → Target Model → Response → Execution Engine → Evaluation.

The Evaluation Engine never communicates directly with external models.

6.10 Adapter Selection Strategy

Execution Engine selects an adapter using configurable rules. Priority order: Explicit user selection, Project default, Organization default, System default. (Future: intelligent routing).

6.11 Standard Execution Request

Every adapter receives the same execution payload.

```
{
  "benchmark_id": "...",
  "run_id": "...",
  "model": "...",
  "prompt": "...",
  "parameters": {
    "temperature": 0.2,
    "max_tokens": 2048
  }
}
```

Provider-specific translation occurs inside the adapter.

6.12 Standard Execution Response

Every adapter returns the same response format.

```
{
  "status": "completed",
  "output": "...",
  "latency_ms": 845,
  "token_usage": {
    "prompt": 612,
    "completion": 388
  },
  "metadata": {}
}
```

6.13 Error Handling

All adapters use common Atlas error codes.

Code	Meaning
ATLAS-EX-001	Connection Failed
ATLAS-EX-002	Authentication Failed
ATLAS-EX-003	Timeout
ATLAS-EX-004	Provider Error
ATLAS-EX-005	Invalid Response

6.14 Security Requirements

Adapters must never: Store user credentials in plain text, Log API keys, Modify benchmark definitions, Modify evaluation results. Secrets shall be retrieved through the Configuration Service.

6.15 Performance Requirements

Metric	Target
Adapter Initialization	< 500 ms
Health Check	< 200 ms
Request Serialization	< 50 ms
Execution Overhead	< 5% of total runtime
<i>(Excludes external model inference time).</i>	

6.16 Testing Strategy

Every adapter shall pass: Unit Tests, Contract Tests, Integration Tests, Failure Simulation, Retry Tests, Timeout Tests. No adapter may be released without passing the Adapter Certification Suite.

Builder Notes

Property	Value
Difficulty	Medium
Owner	Backend Team
Estimated Build Time	2 Weeks
Dependencies	Execution Engine
Prototype Required	Yes
ML Required	No

Engineering Notes

Confirmed Decisions: Atlas Core never communicates directly with providers. Every execution passes through the Adapter Layer. Adapter contracts remain stable. Provider-specific logic is isolated.

Open Questions: Should adapters support capability negotiation? How should streaming responses be standardized? When should intelligent adapter routing be introduced?

CHAPTER 7

Communication & Data Flow Architecture

Document Information

Property	Value
Status	Approved
Engineering Stage	System Communication Design
Priority	Critical
Owner	Platform Architecture Team
Depends On	Chapters 2–6
Related Books	Book 3, Book 4

7.1 Purpose

The purpose of this chapter is to define how data moves throughout the Atlas platform.

Rather than allowing arbitrary communication between services, Atlas adopts a structured communication model that prioritizes reproducibility, maintainability, fault isolation, and future scalability.

Every interaction between components must follow documented communication patterns.

7.2 Communication Principles

1. **No Direct Database Access:** Services may never directly modify another service's database tables. Communication always occurs through APIs or service interfaces.
2. **Single Source of Truth:** Each service owns its own business domain (e.g., Execution Service owns executions).
3. **Stateless Requests:** REST requests shall remain stateless.
4. **Asynchronous Long Tasks:** Operations expected to exceed one second should execute asynchronously (e.g., Benchmark execution).
5. **Immutable Evaluation Data:** Once an evaluation completes, its raw execution data becomes immutable.
6. **Full Traceability:** Every request receives a Correlation ID. Every subsystem propagates that identifier through logs and reports.

7.3 Communication Model

Communication	Technology
User ↔ API	HTTPS / REST
UI ↔ Backend	REST
Services ↔ Services	Internal REST
Background Jobs	Task Queue
Logs	Structured Logging
Metrics	Metrics Service

7.4 Request Lifecycle

User → Dashboard → API Gateway → Authentication → Benchmark Service → Execution Service → Execution Adapter → Target Model → Execution Service → Evaluation Service →

Reporting Service → Database → Dashboard

7.5 Data Flow Architecture

Data Type	Owner
Benchmark Data	Benchmark Service
Execution Data	Execution Service
Evaluation Data	Evaluation Service
Report Data	Reporting Service
User Data	Authentication Service

7.6 Synchronous Communication

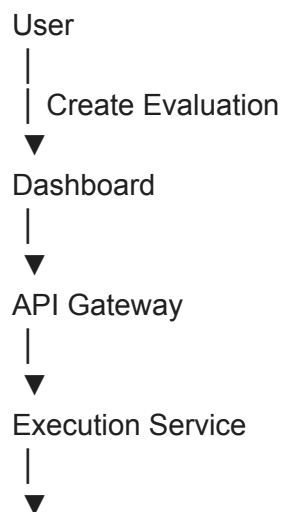
REST APIs are used when: Immediate response is required, Validation occurs, Authentication occurs, Configuration is retrieved. Examples: Create Benchmark, Login, Get Report.

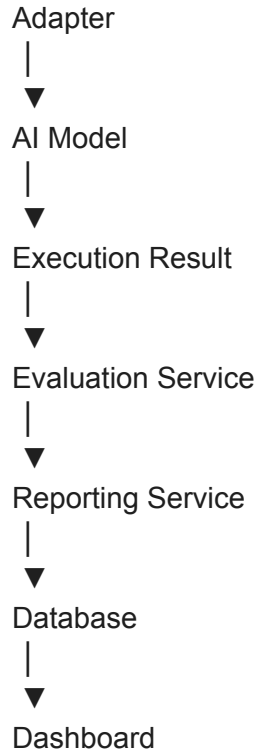
7.7 Asynchronous Communication

Background workers execute long-running tasks. Examples: Benchmark Execution, Report Generation, Large Dataset Validation.

Workflow: Request → Queue → Worker → Database → Notification → Dashboard Update.

7.8 Sequence Diagram – Benchmark Evaluation





7.9 State Machines

- **Benchmark State:** Draft → Validated → Published → Archived
- **Execution State:** Created → Queued → Running → Completed → Evaluated (Failures: Cancelled, Timeout, Failed)
- **Report State:** Requested → Building → Completed → Archived

7.10 Correlation IDs

Every request shall receive a globally unique Correlation ID (e.g., ATL-RUN-2026-000194). Used for Logging, Debugging, Audit Trails, Report Generation, Performance Monitoring.

7.11 Error Propagation

Every service returns standardized responses. Errors must never expose internal implementation details.

```
{  
  "success": false,  
  "error_code": "ATLAS-EV-004",  
  "message": "Evaluation failed.",  
  "correlation_id": "ATL-RUN-2026-000194"  
}
```

7.12 Retry Strategy

Automatic retries are allowed only for transient failures (Network interruption, Provider unavailable). Retries are not permitted for Invalid benchmark, Authentication failure, or Invalid configuration.

7.13 Logging Architecture

Atlas adopts structured logging. Each log entry contains: Timestamp, Service, Correlation ID, User ID, Event, Severity, Duration.

7.14 Observability

Every service exposes: Health endpoint, Metrics endpoint, Readiness endpoint.

7.15 Performance Targets

Metric	Target
API Response	<250 ms
Benchmark Validation	<1 s
Queue Scheduling	<500 ms
Report Build	<5 s
Dashboard Refresh	<1 s

7.16 Security in Communication

All communication must satisfy: HTTPS encryption, JWT authentication, Input validation, Output sanitization, Rate limiting, Audit logging.

Builder Notes

Property	Value
Difficulty	Medium
Owner	Backend Team

Estimated Build Time	2 Weeks
Prototype Required	Yes
ML Required	No

Engineering Notes

Confirmed Decisions: REST is the primary protocol. Long-running tasks execute asynchronously. Correlation IDs are mandatory. Services own their own data.

Open Questions: When should Atlas transition to event-driven messaging? Should report generation become fully asynchronous?

CHAPTER 8

Database Architecture & Data Model

Document Information

Property	Value
Status	Approved
Engineering Phase	Data Architecture
Priority	Critical
Owner	Backend & Database Team
Depends On	Chapters 2–7
Related Books	Book 3, Book 4

8.1 Purpose

The Atlas database is the single source of truth for all persistent platform information. It stores: Users, Projects, Benchmarks, Executions, Evaluation Results, Reports, Datasets, Audit Logs, Configuration. The database is not responsible for business logic.

8.2 Design Philosophy

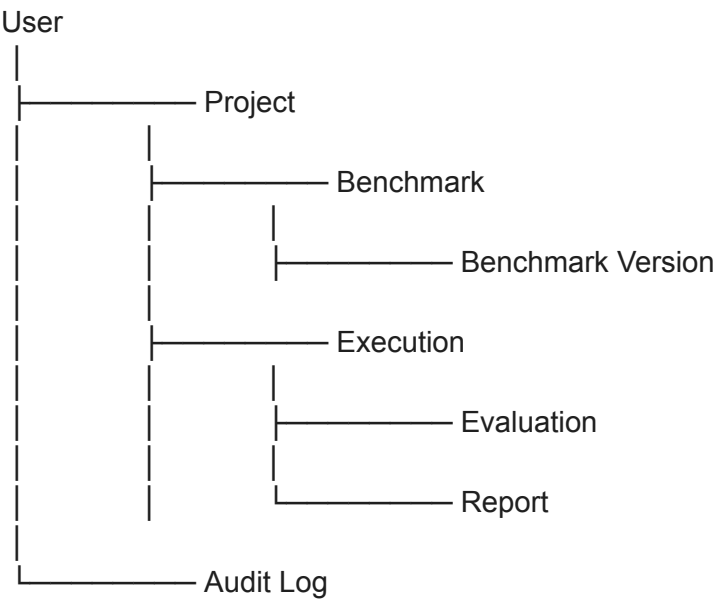
1. **Every table has one owner.** No shared ownership.
2. **Immutable evaluation history.** Completed evaluations are never edited.

- 3. **Everything is versioned.** Benchmarks, Reports, Datasets, Configuration.
- 4. **Normalization first.** Denormalization only for performance.
- 5. **Soft delete whenever historical information matters.**
- 6. **Every important row is auditable.**

8.3 Database Technology

Component	Technology
Relational Database	PostgreSQL
Cache	Redis
Object Storage	MinIO / S3
ORM	SQLAlchemy
Migration Tool	Alembic

8.4 High-Level Entity Model



8.5 Core Tables

Table	Purpose
-------	---------

users	User accounts
projects	Project organization
benchmarks	Benchmark definitions
benchmark_versions	Immutable benchmark revisions
executions	Execution records
evaluations	Evaluation results
reports	Generated reports
datasets	Dataset registry
adapters	Execution adapters
audit_logs	Platform history
configurations	Platform settings

(Sections 8.6 - 8.16 detail individual table schemas as defined in the source).

8.17 Relationships

User → Project → Benchmark → Benchmark Version → Execution → Evaluation → Report

8.18 Indexing Strategy

- **Primary indexes:** UUID
- **Secondary indexes:** project_id, benchmark_id, created_at, status
- **Composite indexes:** benchmark + version, project + execution, user + project

8.19 Transactions

Atlas uses ACID transactions. Required for Benchmark publication, Evaluation completion, Report generation. Large background operations shall use smaller transaction boundaries.

8.20 Backup Strategy

Daily backups. Point-in-time recovery. Monthly archive snapshots. (Future: Cross-region replication).

8.21 Migration Strategy

Every schema modification shall be performed through Alembic. Rules: No manual SQL.

Version controlled migrations. Rollback scripts mandatory.

8.22 Performance Targets

Metric	Target
Benchmark lookup	<50 ms
Report lookup	<100 ms
Evaluation insert	<200 ms
Audit insert	<20 ms

Builder Notes

Property	Value
Database	PostgreSQL
ORM	SQLAlchemy
Migration	Alembic
Estimated Build	2 Weeks
Owner	Backend Team

Engineering Notes

Confirmed Decisions: PostgreSQL is the authoritative datastore. Evaluation history is immutable. Every important entity is versioned. Object storage is separated from relational data.

CHAPTER 9

Security Architecture

Document Information

Property	Value
Status	Approved

Engineering Phase	Security Architecture
Priority	Critical
Owner	Security & Backend Team
Depends On	Chapters 1–8
Related Books	Book 4 (Implementation), Book 5 (Governance)

9.1 Purpose

Security is a foundational architectural concern rather than an optional feature. Atlas stores sensitive assets requiring protection against unauthorized access, tampering, data loss, and misuse.

9.2 Security Objectives

- **SEC-001:** Protect user identities.
- **SEC-002:** Protect execution credentials.
- **SEC-003:** Protect benchmark integrity.
- **SEC-004:** Ensure report authenticity.
- **SEC-005:** Provide complete auditability.
- **SEC-006:** Maintain reproducible evaluation history.

9.3 Security Principles

1. **Least Privilege:** Every component receives only the permissions it requires.
2. **Defense in Depth:** Multiple security controls protect every critical resource.
3. **Zero Trust Between Services:** Internal services authenticate and authorize requests.
4. **Secure by Default:** Security should require explicit opt-out rather than opt-in.
5. **Immutable Audit Trails:** Audit records cannot be modified after creation.
6. **Secret Isolation:** Secrets are never hardcoded into source code or configuration files.
7. **Versioned Security Policies:** Security policies evolve through documented ADRs.

9.4 Authentication

Atlas uses JWT-based authentication for Version 1.

User → Login Request → Authentication Service → Credential Verification → JWT Generation
→ Dashboard → Authenticated Requests

9.5 Authorization

Atlas implements Role-Based Access Control (RBAC).

Role	Permissions
Administrator	Full platform access
Benchmark Author	Create and manage benchmarks
Researcher	Execute benchmarks and view reports
Developer	Manage projects and executions
Viewer	Read-only access

9.6 API Security

All external APIs shall enforce: HTTPS/TLS encryption, JWT validation, Request validation, Rate limiting, Input sanitization, Output encoding.

9.7 Secret Management

Sensitive information includes API Keys, JWT Signing Keys, Database Passwords.

Rules: Never commit secrets to Git. Store secrets in environment variables or a secrets manager. Rotate secrets periodically. Audit secret access.

9.8 Data Protection

Classification	Examples
Public	Public benchmark metadata
Internal	Evaluation reports, project data
Sensitive	Credentials, tokens, password hashes

Sensitive data must always be encrypted at rest where appropriate and encrypted in transit.

9.9 Threat Model

Potential threats include Unauthorized access, Credential theft, API abuse, Denial of Service, Prompt injection affecting benchmark execution. Each threat shall have documented mitigation strategies.

9.10 Audit & Logging

Every security-sensitive action generates an audit event. Audit logs are append-only.

9.11 Security Checklist

Before any release: Authentication tested, Authorization verified, Secrets externalized, Dependencies scanned, Containers updated, TLS enabled, Audit logging enabled, Backup verified.

Builder Notes

Property	Value
Difficulty	Medium
Owner	Backend Team
Build Order	Before Public Release
Dependencies	Authentication Service

Engineering Notes

Confirmed Decisions: JWT authentication, RBAC authorization, HTTPS everywhere, Immutable audit logs, Secrets externalized.

CHAPTER 10

Architecture Decision Records (ADR)

Document Information

Property	Value
Status	Approved
Engineering Phase	Governance
Priority	Critical
Owner	Architecture Team

10.1 Purpose

Architecture Decision Records (ADRs) capture the major engineering decisions made during the design of Atlas.

- **ADR-001 — Atlas is Evaluation Infrastructure:** Atlas is an AI Evaluation Operating System rather than a benchmark collection or leaderboard.
- **ADR-002 — Modular Layered Architecture:** Adopt a layered architecture separating presentation, business logic, infrastructure, and execution adapters.
- **ADR-003 — Execution Adapter Pattern:** All communication with AI models occurs through standardized adapters.
- **ADR-004 — PostgreSQL as Primary Datastore:** Use PostgreSQL as the authoritative relational database.
- **ADR-005 — Redis for Ephemeral Data:** Use Redis for caching and background task coordination.
- **ADR-006 — Docker-First Development:** All development environments shall be containerized.
- **ADR-007 — Local-First Evaluation:** Support local models through Ollama or compatible runtimes as a first-class execution method.
- **ADR-008 — Immutable Evaluation Results:** Completed evaluations cannot be edited.
- **ADR-009 — Version Everything:** Benchmarks, reports, datasets, and configurations are versioned artifacts.
- **ADR-010 — REST for Version 1:** Use REST APIs for synchronous communication.
- **ADR-011 — Asynchronous Execution:** Long-running operations execute through background workers.
- **ADR-012 — Open-Source First:** Prefer mature open-source technologies whenever practical.

10.2 ADR Lifecycle

Proposal → Technical Discussion → Prototype (if required) → Approval → Implementation → Review

Builder Notes

Property	Value
Difficulty	Low
Owner	Architecture Team
Maintenance	Continuous

CHAPTER 11

Architecture Evolution & Technical Roadmap

Document Information

Property	Value
Status	Approved
Engineering Phase	Architecture Governance
Priority	High
Owner	Chief Architect
Depends On	Chapters 1–10

11.1 Purpose

Architecture is not static. Atlas is designed as an evolving engineering platform capable of supporting future evaluation paradigms without requiring large-scale redesign.

11.2 Version Strategy

Version	Goal
Atlas 1.0	Core Evaluation Platform
Atlas 2.0	Community Benchmarks
Atlas 3.0	Intelligent Evaluation
Atlas 4.0	Enterprise Platform
Atlas 5.0	Global Evaluation Infrastructure

11.3 Version 1 Objectives

Focuses exclusively on proving the evaluation pipeline: Benchmark management, Execution engine, Adapter framework, Evaluation engine, Reporting, Dashboard, Authentication.

11.4 Version 2 Objectives

Primary additions: Community benchmark publishing, Reputation system, Benchmark reviews, Public leaderboards, Benchmark discovery, Collaboration. Architecture remains unchanged.

11.5 Version 3 Objectives

Research-oriented features: Adaptive benchmarks, Intelligent benchmark generation, LLM-assisted evaluation, Confidence estimation, Benchmark health analysis.

11.6 Version 4 Objectives

Enterprise capabilities: Organizations, Teams, RBAC expansion, Compliance, Audit exports, Enterprise reports, Private benchmark repositories.

11.7 Version 5 Objectives

Long-term vision: Distributed benchmark execution, Federated benchmark registry, Marketplace, Plugin ecosystem, Multi-region deployments, Academic research portal.

11.8 Architectural Stability

The following components should remain stable across versions: Benchmark Engine, Execution Engine, Evaluation Engine, Reporting Engine, Adapter Framework, Database ownership model, Service contracts.

11.9 Deprecation Policy

Every deprecated feature shall: remain supported for one major release, generate migration warnings, provide migration documentation, include replacement guidance.

11.10 Technology Evolution

Technology choices may evolve (e.g., Docker Compose → Kubernetes). The logical architecture remains unchanged.

11.11 Success Criteria

The architecture is considered successful if: New benchmark types require minimal core changes, New execution providers integrate through adapters, New reports require no redesign of evaluation, Historical benchmark results remain reproducible, Future features build upon existing interfaces.

Engineering Notes

Confirmed Decisions: Atlas evolves incrementally. Core architecture remains stable. Future capabilities extend rather than replace the platform.